CODEMIND

AI CODING MENTOR · MEMORY ENHANCED

Project Documentation

Version 1.0 · March 2026

Live Demo: https://codemind-tau.vercel.app/
GitHub: https://github.com/balapraharsha/CodeMind
Demo Video: Watch on Google Drive

1. Executive Summary

CodeMind is a full-stack AI-powered coding mentor designed to help developers learn smarter and faster. Unlike generic coding platforms, CodeMind remembers every mistake a user makes and uses that memory to craft deeply personalised hints, feedback, and problem recommendations — all powered by AWS Bedrock and the Hindsight memory engine.

The platform addresses a core gap in developer education: most tools treat every session as isolated. CodeMind builds a persistent memory profile for each user, recalling past patterns to warn them about likely mistakes before they make them — transforming every coding session into a targeted learning experience.

Key Highlights

- AI-powered problem generation tailored to the user's weak areas
- Hindsight memory engine — retains and recalls past mistakes across sessions
- Before/After memory toggle — compare personalised vs. generic AI responses
- Real-time code execution with test case validation
- AI feedback on submissions with detailed mistake classification
- Personalised hints powered by recalled memory context
- Global leaderboard with score tracking
- Works without login — accessible to all users as guest
- Full-stack deployment: React/Vite frontend on Vercel, FastAPI backend on Render

2. Problem Statement

Most coding practice platforms provide the same experience to every user regardless of their history. A student who consistently makes null-check errors receives the same generic hints as someone who struggles with time complexity — even after many sessions on the platform. This lack of memory means users repeat the same mistakes indefinitely.

CodeMind directly solves this by building a persistent memory profile using the Hindsight vector memory engine. Every solved and attempted problem contributes to the user's profile, and every hint, feedback, and problem recommendation is augmented with relevant recalled memories.

Core Problems Solved

- No memory across sessions — platforms forget you the moment you close the tab
- Generic hints — advice that ignores the user's specific recurring mistakes
- No personalisation — same problems suggested regardless of your weak areas
- No skill tracking — no visibility into patterns of errors over time
- No adaptive difficulty — problems don't adjust to where you struggle

3. Features & Functionality

3.1 Problem Dashboard

The main hub where users discover and select coding problems. Features include:

- Curated problem library with difficulty badges: Easy, Medium, Hard
- Filter by difficulty and full-text search by title or topic
- AI Learning Insights panel showing weak areas and common mistakes
- Animated stat cards showing problems count, language support, memory status
- AI Problem Generator panel for on-demand AI-generated problems

3.2 AI Problem Generator

Generates entirely new coding problems tailored to the user's weakness profile:

- Select topic (Arrays, Strings, Loops, Linked Lists, Dynamic Programming, Stacks)
- Select difficulty (Easy, Medium, Hard)
- Backend calls AWS Bedrock to generate a unique problem in structured JSON
- Hindsight memory recall targets the user's documented weak areas
- Fallback problem served if AI generation fails — UI never breaks
- Generated problem includes title, description, examples, test cases, and starter code in Python, JavaScript, and Java

3.3 Code Editor & Execution

A full in-browser coding environment powered by Monaco Editor:

- Monaco Editor — the same editor used in VS Code
- Python, JavaScript, and Java language support with syntax highlighting
- Real-time code execution against test cases on the server
- Per-test-case results with expected vs actual output comparison
- Progress ring showing percentage of tests passed
- Anti-cheat: tab switch detection with warning count
- Countdown timer for timed practice sessions

3.4 Memory-Powered Hints

The core differentiator of CodeMind — hints enhanced by recalled memory:

- Before/After Memory toggle for side-by-side comparison
- Memory ON: Hindsight recalls past mistakes relevant to the current problem
- AI mentor proactively warns about likely errors based on user history
- Memory OFF: generic hint with no personalisation
- One-click hint refresh for a different perspective

3.5 AI Submission Feedback

After submitting a solution, the AI provides detailed analysis:

- Mistake classification: null_check, index_error, time_complexity, syntax_error, edge_case, logic_error
- Three-metric AI score: Correctness, Efficiency, Style (each out of 100)
- Personalised feedback using recalled memory context
- Feedback stored back into Hindsight memory for future use
- Leaderboard update on every submission

3.6 Leaderboard

- Global ranking by score (10 points per problem solved + accuracy bonus)
- Per-user stats: problems solved, accuracy percentage, total sessions
- Current user highlighted in the rankings

3.7 Memory Profile

A dedicated page for reviewing the user's learning history:

- INSIGHTS tab: weak areas, common mistakes, AI recommendation, progress bars
- HINDSIGHT tab: cloud memories stored via the Hindsight API
- HISTORY tab: local session attempt history with timestamps and scores

4. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	Fast, modern UI with component architecture
Code Editor	Monaco Editor	VS Code-grade in-browser editing experience
Styling	Custom CSS3 + Tailwind	Dark theme, responsive, branded design
Backend	Python + FastAPI	RESTful async API server
AI Vision	AWS Bedrock — Claude Haiku	Problem generation and feedback analysis
Memory	Hindsight API	Vector memory: retain, recall, persist across sessions
Code Runner	Python asyncio subprocess	Sandboxed multi-language code execution
Frontend CDN	Vercel	Frontend deployment and hosting
Backend CDN	Render	Backend API deployment (free tier)

5. System Architecture

5.1 High-Level Architecture

CodeMind follows a three-tier architecture with a dedicated AI and memory layer:

- Presentation Layer: React SPA hosted on Vercel
- Application Layer: FastAPI REST API hosted on Render
- AI Layer: AWS Bedrock Claude Haiku for problem generation and feedback
- Memory Layer: Hindsight API for vector memory retain and recall
- Execution Layer: Async subprocess runner for Python, JavaScript, and Java

5.2 Data Flow — AI Problem Generation

The following steps describe how a problem is generated end-to-end:

1. User selects topic and difficulty in the AI Generator panel
2. Frontend calls POST /generate-problem with user_id, topic, difficulty
3. Backend calls Hindsight recall to fetch user's past mistake patterns
4. Weakness context is injected into the prompt sent to AWS Bedrock
5. Claude generates a unique problem as structured JSON
6. Backend parses, validates, and returns the problem with starter code
7. React renders the problem card — user clicks Solve to enter the editor

5.3 Data Flow — Memory-Powered Hint

8. User clicks Smart Hint while viewing a problem in the editor
9. Frontend calls POST /hint with user_id, problem details, current code, and memory_mode=true
10. Backend calls Hindsight recall for memories tagged with this user
11. Recalled memories injected into the system prompt as context
12. Bedrock returns a personalised hint that warns about the user's known patterns
13. Frontend renders hint with personalised badge and memory-indicator UI

5.4 Project Structure

```
CodeMind/
├── backend/
│   ├── main.py              (FastAPI app — all routes)
│   ├── hindsight_client.py  (Hindsight API wrapper)
│   ├── problems.json        (Curated problem library)
│   └── requirements.txt
└── frontend/
    ├── src/
    │   ├── pages/           (Dashboard, ProblemSolver, MemoryProfile,
Leaderboard, AuthScreen)
    │   ├── api.js           (Centralised API helper using VITE_API_URL)
    │   ├── App.jsx          (Root with nav, memory toggle, auth state)
    │   ├── icons.jsx        (Custom SVG icon set)
    │   └── index.css        (Global dark theme, animations, component
styles)
    └── vite.config.js       (Vite config with proxy for local dev)
```

6. API Reference

6.1 Authentication

Method	Endpoint	Auth	Description
POST	/signup	None	Create a new user account
POST	/login	None	Login and receive session token
POST	/guest	None	Create a temporary guest session

6.2 Problems

Method	Endpoint	Auth	Description
GET	/problems	None	Get full curated problem library
POST	/generate-problem	None	Generate AI problem tailored to user
GET	/daily-challenge	None	Get today's daily challenge problem

6.3 Code Execution & Submission

Method	Endpoint	Auth	Description
POST	/run-code	None	Execute code against test cases
POST	/submit	None	Submit solution — AI feedback + memory store
POST	/hint	None	Get memory-powered or generic hint
POST	/optimize	None	Get code optimization and complexity analysis

6.4 Memory & Analytics

Method	Endpoint	Auth	Description
GET	/learning-insights/:user_id	None	Get AI-synthesised learning insights
GET	/memory/:user_id	None	Get Hindsight memories and local history
GET	/attempts/:user_id	None	Get last 50 attempt records
GET	/leaderboard	None	Get top 20 users by score
POST	/timer	None	Start or stop practice session timer
GET	/status	None	Get API health and feature status

7. AI Integration

7.1 AWS Bedrock — Claude Haiku

All AI-powered features are driven by AWS Bedrock's Claude Haiku model. The model was chosen for its speed, cost-efficiency, and strong performance on structured JSON generation tasks.

Model: anthropic.claude-haiku-20240307-v1:0
Provider: Anthropic via AWS Bedrock
Max Tokens: 1024–1200 (per call type)
Temperature: Default (0.0 for problem generation)
Region: us-east-1
Timeout: 30 seconds

7.2 Hindsight Memory Engine

Hindsight is a vector memory API that allows CodeMind to retain and recall user-specific memories across sessions. It is the backbone of the personalisation layer.

- retain() — called on every submission to store problem, result, and mistake types
- recall() — called before generating hints, feedback, or problems to retrieve relevant memories
- list_memories() — called on the Memory Profile page to display all stored memories
- Memories are tagged by user_id for isolation between users
- Memories persist across sessions — the AI gets smarter with every interaction

7.3 How Hindsight Memory Is Used

Hindsight memory is deeply integrated throughout the entire CodeMind pipeline — not just as a data store, but as active context that shapes every AI response the user receives.

Memory Retention — After Every Submission

When a user submits a solution, CodeMind calls hindsight_client.retain() to store the following as a semantic memory entry:

- The problem topic and difficulty
- Whether the solution passed or failed
- Mistake types classified by the AI (e.g. null_check, index_error, time_complexity)
- The AI's feedback summary for that submission

These memories accumulate over time to form a rich, searchable profile of the user's coding patterns and recurring errors.

Memory Recall — Before Every AI Response

Before CodeMind asks AWS Bedrock to generate any AI response, it first calls hindsight_client.recall() to retrieve the most relevant past memories for the current user and context. This recall result is injected into the system prompt so the AI is aware of what the user has struggled with historically.

- Problem Generation: Recalled weak areas guide Claude to generate problems targeting the user's gaps
- Hints: Claude is instructed to proactively warn the user about their known mistake patterns
- Submission Feedback: Claude references past mistakes to provide contextual, non-repetitive advice
- Learning Insights: All stored memories are synthesised into an actionable progress report

Before/After Memory Toggle

The Before/After toggle in the UI allows users to directly compare AI responses with and without memory context:

- Memory ON: The hint is generated with recalled past mistakes injected into the prompt — personalised and predictive
- Memory OFF: The hint is generated with a generic prompt and no user history — standard advice any user would receive

This comparison makes the value of Hindsight memory immediately tangible and demonstrable.

Memory Storage Architecture

Hindsight uses a vector database internally, meaning memories are stored as semantic embeddings. When recall() is called with a query, it performs a similarity search to retrieve the most contextually relevant memories — not just the most recent ones.

```
# Retain a memory after submission
await hindsight_client.retain(
    user_id=user_id,
    content=f'Problem: {topic} | Result: {result} | Mistakes: {mistake_types}',
    metadata={'topic': topic, 'difficulty': difficulty}
)

# Recall relevant memories before generating a hint
memories = await hindsight_client.recall(
    user_id=user_id,
    query=f'{problem_title} {topic}',
    top_k=5
)

# Inject into prompt
memory_context = '\n'.join([m['content'] for m in memories])
system_prompt = f'Past mistakes for this user:\n{memory_context}\nGive a
personalised hint.'
```

7.4 Prompt Engineering

Each AI feature uses a carefully crafted prompt to maximise output quality:

- Problem Generator: instructs Claude to return strictly structured JSON with no markdown, no prose
- Feedback: includes recalled memory context, mistake types, and test results
- Hints: injects past mistake patterns and instructs Claude to warn proactively
- Insights: synthesises memory entries into actionable learning recommendations
- All prompts include explicit JSON-only instructions to prevent parse failures
- Fallback JSON is served if parsing fails — the UI never shows a raw error

8. Setup & Deployment Guide

8.1 Prerequisites

- Python 3.11+
- Node.js 18+
- AWS account with Bedrock access enabled in us-east-1
- Hindsight API key from hindsight.vectorize.io

8.2 Environment Variables

Create backend/.env with the following variables:

AWS_BEDROCK_API_KEY: Your AWS Bedrock long-term API key
AWS_ACCESS_KEY_ID: AWS IAM access key (alternative auth)
AWS_SECRET_ACCESS_KEY: AWS IAM secret key (alternative auth)
AWS_REGION: us-east-1
HINDSIGHT_API_KEY: Your Hindsight API key
HINDSIGHT_BASE_URL: https://api.hindsight.vectorize.io
HINDSIGHT_BANK_ID: CodeMind (your memory bank name)

Create frontend/.env.production with:

VITE_API_URL: https://your-render-url.onrender.com

8.3 Local Development

Backend

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload
```

Frontend

```
cd frontend
npm install
npm run dev
```

8.4 Production Deployment

Layer	Technology	Purpose
Frontend	Vercel	Auto-deploys on GitHub push. Set root=frontend, build=npm run build, output=dist
Backend	Render	Set root=backend, runtime=Python 3, start=uvicorn main:app --host 0.0.0.0 --port $PORT
Live URL	codemind-tau.vercel.app	Live demo accessible to anyone

9. Security Considerations

- Passwords hashed using SHA-256 before storage in memory store
- API keys stored as environment variables — never committed to source control
- .gitignore excludes .env and all secret files
- CORS configured to allow all origins for demo; restrict to Vercel URL in production
- No file writes to disk — code execution uses temporary directories cleaned after each run
- Code execution timeout: 8 seconds per test case to prevent resource abuse
- Memory is user-scoped — Hindsight memories tagged by user_id prevent cross-user leakage
- Guest sessions are ephemeral — no data persisted between guest visits

10. Future Scope

10.1 Planned Enhancements

- Persistent database — replace in-memory stores with MongoDB Atlas or Supabase
- JWT authentication — proper token-based auth with refresh tokens
- More languages — C++, Rust, Go support in the code runner
- More problems — expand problem library from 6 to 100+ with difficulty progression
- Streaks & badges — gamification to improve daily engagement
- Mobile app — React Native version for on-the-go practice
- Multi-language UI — support for Telugu, Hindi, Tamil

10.2 Scalability

- Microservices — split code runner into a dedicated containerised service
- Redis caching — cache leaderboard and problem lookups
- Queue-based code execution — job queue for handling concurrent submissions
- Horizontal scaling — multiple backend instances behind a load balancer

11. Conclusion

CodeMind demonstrates how AI memory systems can transform the developer education experience. By combining the intelligence of AWS Bedrock's Claude model with the persistent memory capabilities of Hindsight, we've built a mentor that genuinely gets smarter with every interaction — remembering every mistake, every pattern, and every breakthrough.

The platform is production-ready, deployed, and accessible to anyone through a web browser. The Before/After memory toggle makes the value of AI memory immediately tangible — showing users exactly how much better their experience becomes when the AI knows their history.

This is just the beginning. With planned enhancements around persistent databases, mobile support, and expanded problem libraries, CodeMind has the potential to become the go-to platform for developers who want to train smarter, not just harder.

CODEMIND

https://codemind-tau.vercel.app/ · https://github.com/balapraharsha/CodeMind
Demo Video: Watch on Google Drive